Received: February 4, 2025. Revised: May 5, 2025. 123

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

SadDE: Self-Adaptive discrete Differential Evolution for Combinatorial Optimization Problems

Anisha Radhakrishnan[1] **Jeyakumar G[1]***

[1]*Department of Computer Science and Engineering,*
Amrita School of Computing, Coimbatore, Amrita Vishwa Vidyapeetham, India
* Corresponding author's Email: g_jeyakumar@cb.amrita.edu

Abstract: Differential Evolution (DE) is a stochastic population-based algorithm that tackles continuous optimization problems with decision variables having their values in a continuous domain. The diligent employment of DE also helped to resolve diverse combinatorial optimization problems (COPs), with discrete domains of values for their decision variables, with promising results. However, DE's application in solving permutation-based COPs has not been widely reported in the literature. This paper presents two new DE frameworks with two proposed mapping approaches for solving permutation-based COPs. Firstly, a DE framework named PdDE (Probabilistic discrete Differential Evolution) with a proposed cooperative guided mapping approach is designed and investigated. In PdDE the genes of the mutant vector and selected individuals from the population are selected in a probabilistic method and mapped with the genes of multiple best individuals. This approach improved the searching capability of DE for solving COPs but often resulted in premature convergence. To address this challenge, an extended DE framework named SadDE (Self-adaptive discrete Differential Evolution) with a proposed self-adaptive cooperative guided mapping strategy is designed. The SadDE focuses on developing a guided mapping framework for DE that utilizes a self-adaptive guided strategy to enhance DE's exploration and exploitation capability for solving COPs, suitably. It involves substituting selected genes of the mutant vector and chosen individuals from the population with selected genes from multiple best individuals adaptively. This approach facilitates cooperative guidance of the vectors and directs them towards the optimal solution. The effectiveness of the PdDE and SadDE frameworks is evaluated with 15 instances of the benchmarking travelling salesperson problem (TSP) and 8 instances of the 0/1-Knapsack problem (0/1 KP). Comprehensive empirical and statistical analyses are conducted, comparing PdDE and SadDE against six state-of-the-art DE mapping variants, viz., Truncation process and its variants (TP, TPL, TPI), Rank Based (RB) method, Lowest Rank Value (LRV) method, and Best Matched Value (BMV) method. The experiments are also extended to compare PdDE and SadDE with four well-established metaheuristic algorithms, such as Bee Colony Optimization (BCO), Genetic Algorithm (GA), Firefly Algorithm (FA), and Grey Wolf Optimizer (GWO). On solving the benchmarking TSP instances, the SadDE outperformed the state-of-the-art DE mapping variants by giving the lowest average error percentages of the best solutions. The performance improvements shown by SadDE in comparison with TP, TPL, TPI, RB, LRV, and BMV are 57.49%, 58.28%, 53.82%, 56.74%, 56.79%, and 31.79%, respectively. The comparative study with the metaheuristic algorithms showed that the algorithms integrated with the SadDE mapping approach could outperform the corresponding classical metaheuristic algorithms. The SadDE integrated versions of the BCO, GA, FA, and GWO algorithms demonstrated 29.93%, 28.15%, 32.87%, and 9.14% of performance improvement compared to their classical versions, respectively. These observations proved that the proposed SadDE mapping approach helps to improve the performance of classical optimization algorithms for solving COPs.

Keywords: Combinatorial optimization, Travelling salesman problem, 0/1-Knapsack, Discrete differential evolution, Mapping approaches.

1. Introduction

Differential Evolution (DE) is a robust, simple, yet efficient algorithm in the repertoire of Evolutionary Algorithms (EAs) [1]. DE is a

stochastic population-based approach, inspired by Darwin's selection theory, in which the crossover and mutation operators are search for improved candidate solutions iteratively. Researchers from diverse domains are often confronted with the challenges of optimization. Optimization can be defined as a process of discovering the optimal solution to a problem by accounting for its objectives hampered by a set of constraints. Optimization problems are often encountered in every domain of applications, including medical [2], agriculture [3], network design [4,5], control engineering [6], manufacturing [7], cyber security [8, 9], image processing [10,11], feature selection [12], supply chain [13] and cloud computing [14]. Interpretation of an application and structuring its representation are critical steps in discrete optimization [15]. Many industrial applications are discrete problems [16, 17].

Algorithms designed for the continuous domain cannot be extended directly to solve discrete problems due to infeasible solutions and needed repair mechanisms. They do not provide any clear direction in the search space. The classical DE, formulated for tackling continuous domain problems, has earned its wings in solving them [18, 19]. DE came up short when applied to discrete problems by generating infeasible solutions, impeded by an inability to retain good solutions, and it is considered to be inapplicable to solve permutation-based combinatorial problems [20, 21]. Furthermore, DE may lead to premature convergence or stagnation in the optimization process. However, researchers discovered that DE can tackle discrete problems when properly modified. Modification of DE increases the computation time when the complexity of the problem scales up and results in convergence to the local optimal solution. The significant contributions of this paper are outlined as follows.

- Two mapping approaches, named the cooperative guided mapping approach and its self-adaptive version, are proposed.
- Two DE frameworks, named PdDE and SadDE, to experiment with the above two mapping approaches, respectively, are proposed to solve COPs.
- The incorporation of a self-adaptive algorithm within the framework of Differential Evolution (DE) introduces variability in the selection of genes from multiple individuals within the population and fastens the algorithm's convergence property.
- The proposed mapping approach, incorporated in SadDE, effectively explores a diverse spectrum of potential solutions, mitigating premature convergence issues.

The proposed PdDE and SadDE frameworks are assessed empirically on benchmarking instances of the travelling salesman problem (TSP) and the 0/1 knapsack problem (0/1 KP). Their effectiveness is compared with six extant state-of-the-art mapping approaches of DE for COPs. The performance of SadDE is further validated by integrating it with classical metaheuristic algorithms viz., BCO, GA, and GWO. The experimental result shows that the proposed frameworks had promising results.

The remaining section of this paper is organized as follows. Section 2 presents the related work for solving combinatorial optimization problems. Section 3 details the classical Differential Evolution algorithm. Section 4 explains the proposed mapping approaches and the proposed frameworks for discrete DE. Section 5 explains the benchmarking problems used for this study. Section 6 explains the validation of the experimented result achieved for the proposed mapping approaches. Finally, Section 7 discusses the conclusion of the study and future work.

2. Related works

Researchers have made several attempts to augment DE's capabilities to manage discrete optimization problems. The important insight is that the DE's mutation operation results in the generation of mutant vectors with real-valued genes. These gene values need to be modified appropriately to make DE work for COPs. The popular approaches available in the literature for this purpose are categorized as ensemble approaches, operator-based approaches, hybridization approaches, and co-evolution approaches [22]. Ensemble approaches enhance algorithm performance by combining different operators and methods on parallel or single populations. However, they increase computational time and fine-tuning the operators is a challenging task. Operator-based approaches enhance the effectiveness of algorithms by incorporating tweaked mutation and crossover techniques, but these approaches are often problem-dependent. Hybridization combines two or more algorithms. Hybridization can be performed between global metaheuristic algorithms or combined with local search methods. This approach demonstrates better search efficiency; however, it can result in high computational time complexity. In co-evolutionary approaches, the result obtained by a subpopulation is migrated to another subpopulation, leading to the convergence of the optimal solution. However, if

diversity is not properly maintained, it may lead to premature convergence.

In general, the design of algorithms for discrete optimization includes model-based techniques that are based on the ensemble strategy. They are illustrated as strings, trees, categorical variables, graphs, permutations, and ordinal integers. The operator-based approaches are further classified as modified learning and modified structure. The modified learning approach adopts mutation by combining linear algebra and swap-based routines. The modified structure-based approaches include fuzzy matrix and set-based approaches. The additional strategic approaches for solving discrete optimization problems are custom modeling, discrete modeling, structure modeling, naive approach, and mapping approach [23]. Custom modeling is effective for problems where prior knowledge about the application is known. The prime drawback of this approach is its migration to other data structures. This approach does not perform well for black-box problems. The discrete models are adaptable for tree-based models but unsuitable for complex problems that result in infeasible outcomes. Structure modeling performs well for problems that have less complexity and can be avoided for binary, integer, and categorical data representation. A naive approach can effectively solve smaller dimension problems but results in generating infeasible solutions for larger solution spaces. Mapping approaches are good techniques if mapping can be done appropriately. This can be rank-based, position-based, or truncation-based. When the complexity of the problem increases, there is difficulty in performing the mapping approach. The major downside of this approach is redundancy and infeasible solutions.

Kennedy and Eberhart [24] proposed a position-based scheme, a discrete version of particle swarm optimization (PSO) to determine the trajectory of the particles. However, due to the limitation of binary encoding the binary particle swarm algorithm might not find exact solution. Furthermore, its exploration capability is restricted, as it lacks the ability to perform large leaps across the search space, unlike other evolutionary algorithms EAs. Lampinen and Zelinka put forth a truncation method [25], which rounds off the floating point and produces better results. The major drawback of these approaches is constrained to a set of standard values. Onwubolu and Davendra presented a discrete version of DE, including forward and backward transformation of a vector for flow shop scheduling [26]. Forward transformation converts discrete values to floating values, while backward transformation converts floating values to discrete. As the forward and backward transformation approach generates redundant values, a repair mechanism is designed to exclude infeasible solutions. Snyder and Dasking recommended a random key genetic algorithm for solving TSPs [27]. The cities are rearranged based on the smallest position value (SPV) of the generated keys. The proposed approach showed good robustness and performance across different problem instances when appropriately tuned, its performance is sensitive to parameter settings and scalability on instances with large number of cities.

Tasgetiren et al., presented a PSO algorithm where an SPV rule and variable neighborhood search (VNS) are applied for the flow shop sequencing problem [28]. The algorithm effectively adapts PSO for discrete scheduling, achieving good performance for minimizing makespan and flowtime. However, it is sensitive to parameter settings and premature convergence. Lichtblau suggested an order-based method for solutions to set partitioning problems [29]. Although this option improved the performance of discrete DE, high computational latencies are the drawback. Tasgetiren et al., (2009a) proposed the smallest position value to address the generalized TSP (GTSP) [30]. The proposed approach improved the performance but the mutant vector needed re-initialization to keep values within the search space boundary. Tasgetiren et al., (2009b) proposed the Discrete/Binary method as an improved version of the DE algorithm for solving GTSPs [31]. This approach effectively extends DE's search capability and simplicity to tackle combinatorial problems. However, the effectiveness of the approach is sensitive to parameter settings and scalability. Furthermore, it tends to result in premature convergence and increased computational complexity as the problem size grows. He et al., implemented discrete binary DE for selecting the subset of best features [32]. The proposed approach enhanced the classification accuracy and effectively reduced the number of features. Despite these advantages, the approach is sensitive to parameter setting and tends towards premature convergence. Li and Zhang proposed a discrete hybrid DE algorithm to solve integer programming problems, where the initialization of individuals is generated as integers [33]. The proposed algorithm significantly improved solution quality and maintained population diversity, helping to avoid premature convergence. However, the algorithm has high computational time and is sensitive to parameter settings. Mutation produced real values which are modified to integers by a mixed truncation process.

Ali et al., (2015) propounded an improved version of DE, with an order-based approach for

resource-constrained project scheduling problems (RCPSP) [34]. The proposed algorithm demonstrated improved solution quality and faster convergence but, limitation of the algorithm is high computational cost, parameter sensitivity, and premature convergence. Ali et al., (2019) proposed Best-matched value (BMV), an effective mapping technique that swapped the gene of the mutant vector and selected individuals from the population, with the gene of the best individuals [35]. The proposed approach effectively showed improved solution quality achieved a good trade-off between exploration and exploitation. Furthermore, the mapping technique ensures that generated solutions remain feasible. Nevertheless, the algorithm suffers from high computational complexity, premature convergence, and sensitivity to parameter settings. Engelbrecht et al. presented a set-based method for feature selection [36]. The proposed algorithm-maintained population diversity, and furthermore, the set-based representation effectively selected relevant features and achieved better performance than binary encoding. However, the approach is sensitive to parameter settings, and evaluating multiple feature subsets during the optimization process increased computational time, especially with high-dimensional data. Gain and Dey proposed an adaptive position-based crossover in to genetic algorithm for data clustering, which dynamically adjusts crossover points based on data distribution, enhancing the exploration-exploitation balance and improving clustering quality [37]. However, it operates at the operator level without adaptive solution encoding and incurs high computational overhead. This method is also sensitive to population diversity and prone to premature convergence due to overfitting to the current population distribution. Chaves et al., (2024a) proposed the Random-Key Optimizer, a simple encoding method for combinatorial problems that maintains solution feasibility and balances exploration and exploitation [38]. However, the method incurs high computational overhead during decoding and is sensitive to parameter tuning, especially when scaling to very large problems.

Chaves et al. (2024b) proposed the Random-Key GRASP, a continuous representation for combinatorial problems [39]. The proposed method enhanced solution diversity with integration of local search strategies. However, the approach faces challenges such as decoding overhead and inconsistent performance due to the absence of memory mechanisms. Singsathid et al. proposed self-adaptive differential evolution algorithm with dynamic fitness-ranking mutation and pheromone strategy [40]. The proposed method improves search balance by dynamically adjusting mutation based on fitness ranking and guiding exploration through pheromone trails. While it enhances solution quality and population diversity, it does not focus on solution encoding mapping and suffers higher computational costs. Alvarez-Flores et al. proposed a modified discrete differential evolution algorithm [41] that applies specialized mapping techniques and modified operators to effectively solve the TSP, improving solution quality and ensuring feasibility. However, this method results in high computational time and suffers from a lack of population diversity. Zhang et al. proposed DE combined with adaptive neighborhood search strategy [42]. It achieved a better trade-off between exploration and exploitation, especially in large-scale problem instances. However, the algorithm has high computational time. Li and Chen proposed an Improved Discrete Differential Evolution (IDDE) algorithm with Opposition-Based Learning (OBL) to solve the Travelling Salesman Problem [43]. This technique enhances population diversity and avoids premature convergence by integrating OBL. The advantages comprise better exploration and exploitation balance, faster convergence, improved solution quality, and strong performance on standard TSP benchmarks. However, the method has high computational time and highly depend on local search.

Summarizing the foregoing literature, evolutionary algorithms for combinatorial optimization problems can be categorized into operator-level adaptation strategies and mapping strategies. Both these strategies require domain knowledge to effectively solve the discrete problems. As well as, the computational time increases as the complexity of the problems rises. The approaches like binary encoding, random-key encoding, and position-based mappings have contributed in solving specific combinatorial optimization problems. However, they primarily depend on fixed or single mapping schemes. These static solution encoding strategies limit the flexibility of the search process, reduce population diversity over time, and increase the risk of premature convergence, especially in complex or large-scale problems. A major limitation observed in these mapping approaches is the generation of infeasible solutions. An infeasible solution results in local optima and stagnation. Tuning these infeasible solutions appropriately by balancing the exploration and exploitation is a challenging task. This tuning increases the time complexity when dealing with complex problems.

Recent efforts towards self-adaptive DE algorithms have mainly focused on controlling the parameters of the mutation and crossover operators,

with limited attention given to improvements in solution encoding (or mapping). Moreover, most studies in the literature lack a cooperative mechanism to guide the search process by adaptively mapping the solution vectors. To overcome these limitations, this paper proposes a DE framework named SadDE (Self-adaptive discrete Differential Evolution). The SadDE framework addresses the above-mentioned research gaps by employing a cooperative guidance mechanism that adaptively selects genes from multiple best individuals, resulting in better convergence and solution quality.

3. The classical DE algorithm

Storn and Price proposed DE, as a population-based optimizer to the Evolutionary Algorithm (EA) family, suitable for solving continuous domain optimization problems [1]. A common algorithmic template of EA is presented as Algorithm 1.

Similar to other EAs, the DE algorithm also iteratively generates offspring using the mutation and crossover operators. However, DE is simple and robust. The sequence of operations in DE are initialization of population, mutation, crossover, and selection. First, a population of pre-determined size P_s, with each candidate (denoted as i_q, where $q = 1, 2, 3, \dots, P_s$) having D dimensions, is initialized. Then, each individual candidate (or vector) in the population is evaluated by a fitness function. Successively, the population is updated iteratively by the mutation, crossover, and selection operations. The mutation and crossover operators of DE have a major impact on its search capability.

3.1 DE's mutation

DE's mutation is a perturbation process that generates a mutant vector by computing weighted differences of paired random vectors and adding them to a third random vector of the population. Storn and Price proposed a representation for DE variants denoted as DE/x/y/z where x is the mutation operator, y refers to the number of paired vectors, and z refers to the crossover scheme. DE/rand/bin is the primary and commonly used DE variant. The mutant vector of the DE/rand/1/bin variant is generated by Eq. (1).

$$v_i^g = x_{k3}^g + F.\left(x_{k2}^g - x_{k1}^g\right) \tag{1}$$

where v_i^g is the mutant vector for individual i in generation g, (x_{k1}, x_{k2}, and x_{k3}) are mutually exclusive three random vectors selected from the population, and F is the scaling factor (or mutation step size) range between [0, 2]. The value of F controls the mutation process. Smaller mutation step sizes require a longer convergence time for the algorithm and larger mutation step sizes skip the optimal solutions during the evolutionary search process. The F value should be good enough such that it ensures a balance between exploration and exploitation in the population, thereby avoiding premature convergence.

Algorithm 1: Classical DE

```
Begin
Initialize the Population
Evaluate the fitness of each individual
Do While Termination Condition is not met
   For each individual in the population
      Do
         Perform mutation
         Perform crossover
         Perform selection
      End Do
   End For
End While
End
```

3.2 DE's crossover

In order to increase the population diversity, the crossover operator is applied after the mutation. The trial vector is the outcome of merging the target and mutant vectors, with a crossover probability *CR* in the range of [0,1]. The crossover operator has to ensure diversity in the population. The expression for performing the crossover operation is provided in Eq. (2).

$$u_{i,j}^g = \begin{cases} v_{i,j}^g, & if\ \ r(j) \le CR\ or\ j = j_{rand} \\ x_{i,j}^g, & otherwise \end{cases} \tag{2}$$

where $u_{i,j}^g$ is the component of the trial vector for individual *i* at the dimension *j* in the generation *g*, the $v_{\mathrm{i,j}}^{\mathrm{g}}$ is the mutant vector component, the $x_{i,j}^g$ is the target vector component, the $r(j)$ is a uniformly distributed random number in [0,1] for each dimension j, the *CR* is the crossover probability, and the j_{rand} is to ensure that at least one dimension from the mutant vector is used in the trial vector.

3.3 DE's selection

The selection operation determines whether a trial vector or a target vector survives for the next generation, using the Eq. (3).

$$x_i^{g+1} = \begin{cases} u_i^g, & if\ f(u_i^g) \leq f(x_i^g) \\ x_i^g, & otherwise \end{cases} \quad (3)$$

4. The proposed frameworks

This paper proposes two DE frameworks named PdDE and SadDE incorporated with two proposed mapping approaches named 'cooperative guided mapping approach' and 'self-adaptive cooperative guided mapping approach', respectively, to solve COPs.

(1) *Probabilistic discrete Differential Evolution (PdDE)* - This framework employs a collaborative guided mapping strategy to probabilistically map genes from multiple vectors to enhance the search capabilities of DE.

(2) *Self-Adaptive discrete Differential Evolution (SadDE)* - The SadDE is an extended mapping variant of PdDE that utilizes a self-adaptive collaborative guided mapping strategy. This strategy effectively balances the exploration and exploitation aspects of the algorithm, leading to improved performance. The structure of the proposed SadDE is presented in Fig 1. Algorithm 2 represents the pseudocode for SadDE.

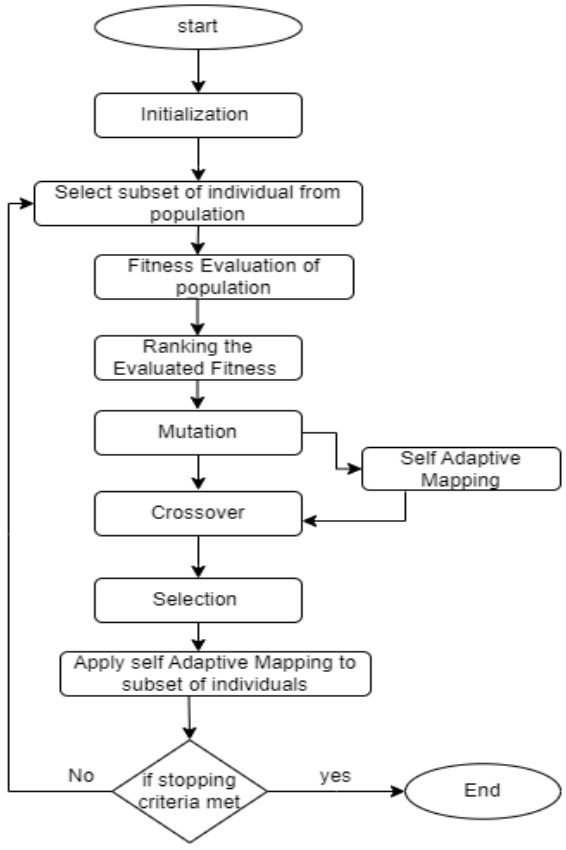

Figure. 1 Structure of Self-Adaptive Discrete Differential Evolution

Table 1. Nomenclature

Notation	Description
D	Dimension
g_{max}	Maximum generation
g	Generation
P_S	Population Size
P	Population
$\grave{P}$	Subset of P
CR	Crossover Rate
δ^{LB}	Lower bound for gene mapping
δ^{UB}	Upper bound for gene mapping
Pr	Uniform probability
α	Best individual
β	Second best individual
φ_{can}	The probability for selecting an individual for mapping from P

For convenient reference, the symbols mentioned in the paper are summarized in Table 1. The proposed *PdDE* and *SadDE* frameworks retain similar methods of population initialization, mutation, crossover, and selection, as in the classical DE, while differing in their mapping strategies.

4.1 Population initialization

Firstly, proposed DE frameworks start the initialization of the population. The population with P_s numbers of candidates is denoted as $P = i_1, i_2, i_3, \ldots, i_{P_s}$.

Algorithm 2: Self adaptive discrete Differential Evolution (SadDE)

Input: Population P, Crossover rate CR, and Max Iteration = g_{max}
Individual Selection for mapping = φ_{can}
Output: Best solution
Initialize population P
  For each individual i in P do
    Evaluate the fitness of i
  End For
  While the termination condition is not met do
    Sort population P based on fitness
    For each individual i in P do
      Generate uniform probability Pr(i) in the range [0, 1]
      If Pr(i) $\leq \varphi_{can}$ then
        Add individual i to P'
      End If
      Generate mutant vector v_i using:
      $v_i^g = x_{best}^g + F(x_{k3}^g - x_{k4}^g) + F(x_{k2}^g - x_{k1}^g)$
      Apply Self adaptive Mapping (refer Algorithm 4)
    End For
    For each individual i in $\grave{P}$ do

```
    Apply Self adaptive Mapping (refer Algorithm 4)
  End For
  Replace the individual at position g in P
    with the mapped individual i in P̀
  For each individual i in P do
    For each dimension j do
      Generate random number r(j)
      If r(j) ≤ CR or j == j_rand then
        u_{i,j}^g = v_{i,j}^g
      Else
        u_{i,j}^g = x_{i,j}^g
      End If
    End For
    Apply greedy selection:
    If f(u_i^g) ≤ f(x_i^g) then
        x_{i,j}^{g+1} = u_i^g
    Else
        x_{i,j}^{g+1} = x_i^g
    End If
  End for
End While
```

The population is initialized as per the problem chosen to solve. For solving the Travelling Salesman Problem (TSP), each individual i_q in the population is defined as a permutation of cities, i = 1, 2, 3, . . ., n, where n is the number of cities. The selection of cities for each individual i_q is a random process. Let C_i represents the set of cities for individual i_q, where C_i is a random permutation of $i = \{1,2,3,\ldots,n\}$. The objective function, defined as $f: P \rightarrow R$, evaluates the individual's fitness and they are ranked based on their fitness values. Individuals with the best fitness ranked first, and those with the worst fitness ranked last. Two individuals in the population, α and β, are selected for mapping. The working of the random selection function to choose α and β for PdDE and SadDE is mentioned in Section 4.4. A uniform probability (*Pr*) distribution determines the selection of individuals from the population for the mapping approach. The $\grave{P}$ denotes the set of selected individuals such that $\grave{P} \subseteq P$. Each individual i_q is selected if $Pr(i) \leq 0.3$.

4.2 Mutation

In the proposed PdDE and SadDE frameworks, the *DE/best/2* mutation strategy is used for mutant vector generation. The *DE/best/2* is superior among DE mutation strategies due to its ability to maintain population diversity [1]. The mutation scale factor (F) plays a major role: F > 1 can solve many problems, while F ≤ 1 ensures faster results. The best vector from the population is designated as the base vector and four mutually exclusive random vectors are selected from the population. The weighted difference of two random vectors is added with the weighted difference of another two vectors and the base vector. The mutant vector for *DE/best/2* is generated using Eq. (4).

$$v_i^g = x_{best}^g + F\left(x_{k3}^g - x_{k4}^g\right) + F\left(x_{k2}^g - x_{k1}^g\right) \quad (4)$$

where $x_{k1}, x_{k2}, x_{k3}, x_{k4}$ are mutually exclusive random vectors selected from the population. The genes of the mutant vector produced by the mutation operator are continuous numbers. It is converted to an integer by ranking the gene values of the mutant vector representing the city nodes in TSP. The mutant vector v_i and individuals in $\grave{P}$ are mapped with genes of α and β using the mapping approach explained in Section 4.4.

4.3 Crossover and selection

Binomial crossover is performed, after applying the mapping approach, to generate the trial vectors by mixing the target vectors from the population and the mutant vectors. The fitness of the trial vectors is evaluated using the objective function. Selection is performed by comparing the fitness of the trial vector and the target vector. Greedy selection is performed, where the individual with the better fitness is selected for the next generation. The evolutionary process gets terminated when 95% of the population has similar fitness values, in order to reduce the computational time.

4.4 The mapping approaches

This subsection explains the proposed cooperative guided mapping approach and the self-adaptive mapping approach incorporated in the PdDE and SadDE frameworks of DE, respectively, in detail.

4.4.1. The cooperative guided mapping approach of PdDE framework

The Algorithm 3 presents the mapping approach of PdDE. After the mutant vector is converted to an integer vector by ranking them, a uniform probability *Pr* is generated for the gene selection of mutant vectors. The $\delta^{LB} = 0.5$ and $\delta^{UB} = 0.6$ are set based on an empirical study, which is presented in Section 6. Genes with a probability greater than or equal to the upper bound δ^{UB} are replaced with genes of the first best individual in the current population, which is denoted as α as mentioned in Eq. (5).

Algorithm 3: PdDE mapping

Step 1: Mutation to generate mutant vector

For each gene j in mutant vector v_i do
  Generate uniform random probability *Pr(j)* ∈ [0, 1]
  Set δ^{LB} = 0.5
  Set δ^{UB} = 0.6
  If *Pr(j)* ≤ δ^{LB} then
    Swap gene of individual β into position j
  Else If *Pr(j)* ≥ δ^{UB} then
    Swap gene of individual α into position j
  Else
    Keep gene $v_{i,j}$ unchanged
  End If
End For

Step 2: Mapping from population subset $\dot{P}$

For each individual *i* in subset $\dot{P}$ do
 For each gene *g* in individual *i* do
  If *Pr(j)* ≤ δ^{LB} then
   Swap gene of individual β into position j
  Else If *Pr(j)* ≥ δ^{UB} then
   Swap gene of individual α into position j
  Else
   Keep gene $v_{i,j}$ unchanged
  End If
 End For
End For

Genes with a probability less than or equal to the lower bound δ^{LB} are replaced with the genes of the second-best individual β which is randomly selected from the top 5 individuals based on the function described in Eq. (6). The Eq. (7) represents the gene mapping process of the proposed cooperative guided mapping approach.

$$\alpha = Top\_1(P) \tag{5}$$

$$\beta = random\_selection(Next\_Top\_5(P - \alpha)) \tag{6}$$

$$v_i' = \begin{cases} \beta_{i,j} & if\ \Pr(j) \le \delta^{LB} \\ \alpha_{i,j} & if\ \Pr(j) \ge \delta^{UB} \\ v_{i,j} & Otherwise \end{cases} \tag{7}$$

where Top_1( ) selects the best individual from the population and Next_Top_5( ) selects the top 5 individuals next to the best individual.

The mapping approach of PdDE is also applied to selected individuals in $\dot{P}$.

4.4.2. The self-adaptive mapping approach of SadDE

Algorithm 4 presents the self-adaptive mapping approach of SadDE. In this mapping approach, the genes of the mutant vector v_i are mapped with the genes of individuals α and β, which represent two selected candidates. The α for SadDE is selected from the population, as denoted in Eq. (8) and β is selected as mentioned in Eq. (9)

$$\alpha = \text{random\_selection (Top\_10 (P))} \tag{8}$$

$$\beta = \text{random\_selection(Next\_Top\_10 (P} - \alpha)) \tag{9}$$

Algorithm 4: Self Adaptive Mapping

For each gene j in individual v_i do
  θ_1 = random.uniform(0.1, 0.3)
  θ_2 = random.uniform(0.7, 0.9)
  f_x = 0.1, f_z = 0.9, f_y = 0.7
  If generation g ≤ 50 then
    δ^{LB} = random.uniform(0.1, 0.3)
    δ^{UB} = random.uniform(0.7, 0.9)
  Else
    δ^{LB} = (f_x / f_z * g) + θ_1
    δ^{UB} = (f_y / f_z * g) + θ_2
  End If
  Generate random probability Pr(j)
  If Pr(j) ≤ δ^{LB} then
    Swap gene of β
  Else if Pr(j) ≥ δ^{UB} then
    Swap gene of α
  Else
    Keep gene $v_{i,j}$ unchanged
  End If
End For

where, Top_10 denotes the best 10 individuals from the population, and $Next\_Top\_10$ denotes the top 10 from the remaining population. By employing a cooperative-directed strategy, this approach guides the mutant vector in its exploration for reaching an optimal solution. This cooperative-directed strategy leverages the genetic information of α and β to enhance the search process.

For the initial 50 generations, θ_1 and θ_2 are generated as mentioned in Eq. (10) and Eq. (11). Here, θ_1 is randomly generated from the interval [0.1, 0.3] and θ_2 is randomly generated from the interval [0.7, 0.9]. This is to ensure that the values are generated within the boundary values so that the genes of multiple vectors can be mapped along with the genes of the mutant vector. This encoded (or mapped) solution includes genes of the best vectors and the mutant vector, which can generate a quality solution.

$$\theta_1 = random.uniform(0.1, 0.3) \quad (10)$$

$$\theta_2 = random.uniform(0.7, 0.9) \quad (11)$$

This adaptive mapping process is controlled by the parameters δ^{LB} and δ^{UB}. They are dynamically generated according to the formulations presented in Eq. (12) and Eq. (13). These equations determine the lower and upper bounds for the mapping process, ensuring the adaptability of the mapping approach to the problem space.

$$\delta^{LB} = (fx/fz * g) + \theta_1 \quad (12)$$

$$\delta^{UB} = (fy/fz * g) + \theta_2 \quad (13)$$

where *fx*, *fy* and *fz* values are set constant as 0.1, 0.4 and 0.9, respectively, g represents generation. The θ_1 and θ_2 act as stochastic elements in the equation and contributing to the adaptability and exploration capability of the mapping approach. They allow exploration across different regions of the solution space, and their stochastic nature adds randomness, preventing the algorithm falling in local optima. This feature potentially leads to improved solutions.

4.5 The enhancement in the mapping approach of SadDE

This section details the motivation behind extending the cooperative guided mapping approach of PdDE to the self-adaptive mapping approach of SadDE. Upon investigating PdDE, it is observed that selecting the best individuals based on their top rank led to premature convergence within a few iterations. While exploitation is rapid, it did not result in any improvement in exploration after a few iterations. Another significant drawback noticed, while solving the TSPs is "as the generation increases, a minimum number of gene mappings is more effective for the problem instances with fewer city nodes, whereas for the instances with a larger number of cities, more number of gene mappings are required". These findings inspired the development of self-adaptive mapping strategies for SadDE.

In SadDE, δ^{LB} and δ^{UB} are chosen adaptively, as described in Eq. (12) and Eq. (13). Introducing randomness through θ_1 and θ_2 , along with the random selection of α and β, helps to maintain diversity in the discrete differential evolution process. As the generation progresses, a balance in gene mapping from multiple vectors prevents premature convergence. Hence, SadDE could achieve a better trade-off between exploration and exploitation compared to PdDE.

The dynamic adjustment of δ^{LB} and δ^{UB} balances exploration and exploitation by widening the search space early and narrowing it progressively. As generation g increases the search boundary is narrowed to explore wider range of solution. In the initial search, higher values of δ enable broader exploration, while in later stages, smaller δ refines the search around promising regions. This gradual shift is supported by adaptive control of θ_1 and θ_2, maintaining population diversity and convergence behavior.

5. The benchmarking problems

The performance of the proposed DE frameworks incorporated with proposed mapping approaches is tested on two prominent classical combinatorial optimization problems. The experiments include different instances of TSP (Travelling Salesman Problems) and 0/1 Knapsack problem (0/1 KP).

5.1 Travelling salesman problem

The travelling salesman problem is a combinatorial optimization problem that stimulates the researchers' attention. TSP and its variants like multiple travelling salesman problem (MTSP), and asymmetric travelling salesman problem (ATSP) have aroused in interesting use-case applications in diverse domains like networking, designing, and planning [36]. The TSP entails minimizing the total travel cost of a salesman's covering *m* cities and returning to the initial city (starting point), constrained by visiting each city only once. The total cost is calculated as the total distance travelled (*Dist*) using Eq. (14). Thus, the salesperson must visit the cities with the shortest distance possible. The cost matrix of TSP is the minimum distance, mathematically formulated over a graph *G*, where *G = (N, E), N* represents the cities *N* = {1, 2, …, n} and *E* represents the edges *E* = {1, 2, …, e}. The objective function is

$$Minimize\ Dist = \sum_{l=1}^{n} dist_{l,l+1} \quad (14)$$

where $dist_{l,l+1}$ refers to the distance between consecutive cities *l* and *l+1*.

5.2 0/1 Knapsack problem

The Knapsack problem is another classical combinatorial optimization problem. The primary objective of this problem is to maximize the profit. The item *i* is placed in the knapsack without violating

the knapsack capacity *C*. The total number of items that can be placed is *n*. Each item *i* has value *v* ($v > 0$) and weight *w* ($w > 0$). The y_i is a decision vector that represents 1 if the item is picked, if not then 0. The objective of this problem is calculated using Eq. (15). The constraints are given in Eq. (16).

$$Maximize \;\; f(y) = \sum_{i=1}^{n} y_i \, v_i \tag{15}$$

$$\sum_{i=1}^{n} y_i \, w_i \leq C \;\; and \;\; y_i \in \{0,1\} \tag{16}$$

6. Experimental results and analysis

The efficacy of the proposed PdDE and SadDE frameworks and their mapping approaches are empirically substantiated through experimentation on two well-known combinatorial optimization problems.

a. Travelling salesman problem from TSPLIB [44].
b. 0/1 Knapsack problem from https://people.sc.fsu.edu/~jburkardt/datasets/knapsack_01/knapsack_01.html

The numerical experiments are implemented using Python 3 in i7 processor, 8 GB RAM. The PdDE and SadDE are tested on 15 instances of TSP and 8 instances of 0/1 KP and are compared with six state-of-the-art mapping approaches. Empirical results for the state-of-the-art approaches of TSP and 0/1 KP are taken directly from [35]. The primary focus of the experimentation is on encoding solution mapping. No additional local search algorithms are embedded with PdDE and SadDE for this study.

The PdDE and SadDE are compared with the state-of-the-art mapping approaches for DE with the same parameter settings as given in Table 2 to maintain fairness in empirical comparison. The study is further extended by integrating the PdDE and SadDE frameworks with various DE-based mapping approaches, including TP, TPL, TP1, RB, LRV, and BMV, and tested on 5 instances of the TSP, including instances with more than 500 cities.

Additionally, SadDE is applied to enhance four metaheuristic algorithms and they are BCO, GA, FA, and GWO. Initially, these classical metaheuristic algorithms are executed independently on 8 TSP instances, and subsequently, their performance is further investigated by integrating SadDE using the same parameter settings.

Due to the randomization in the evolutionary operators of the algorithms considered, minor variations are observed in the average error of the results. Hence, the experiments are repeated for 30 runs each with 5000 generations, for each problem instance, and the average performances are reported. The mutation factor F is self-adaptive and used as mentioned in [35].

Table 2. Parameter Setting for DE mapping

$\boldsymbol{\theta_1}$	$\boldsymbol{\theta_2}$	$\boldsymbol{\varphi_{can}}$	$\boldsymbol{F}$	$\boldsymbol{CR}$	$\boldsymbol{P_s}$	$\boldsymbol{g_{max}}$
0.1 - 0.3	0.7 - 0.9	0.3	0.6 - 1.6	0.9	100	5000

In addition to the parameters commonly associated with DE (the population size, scaling factor, and crossover probability), the proposed algorithm PdDE introduces three additional parameters: φ_{can}, δ^{LB}, and δ^{UB}. In each iteration, a probability denoted as φ_{can}, is generated for the selection of a subset of individuals $\grave{P}$ from the population for applying the mapping method. The individuals in the population, for them the generated probability is less than φ_{can}, are selected for $\grave{P}$. This ensures only a few individuals are selected for mapping from the population. This is performed to improve the convergence rate. An experimental study on the mapping approach is conducted to analyze the cooperative guided strategy through multiple vectors gene mapping. The performance of these parameters is determined through an analysis of the results obtained for different settings of boundary values and φ_{can}. Once the best parameter values are identified in one experiment, they are subsequently fixed for further investigations.

Initially, an experimental study is conducted to analyse the cooperative guided strategy of multiple vectors in the mapping approach. The parameter value setting for δ^{LB} and δ^{UB} is manually performed and uniform probability is generated for the selection of genes. The selected genes of the mutant vector and selected individuals are mapped with genes of α and β in the population. The upper and lower bound of the probability for mapping are set with three boundaries. For the easiness of explanation (δ^{LB} = 0.3 with δ^{UB}= 0.7) is represented as bv_1, (δ^{LB}= 0.5 with δ^{UB}= 0.6) as bv_2 and (δ^{LB}= 0.4 with δ^{UB}= 0.5) as bv_3. The genes with a probability less than the δ^{LB} and a probability value greater than δ^{UB} are selected for mapping. Selection of δ^{LB} = 0.3 and δ^{UB} = 0.7 indicates that 30% of genes from the lower bound and 30% of genes from the upper bound are selected for mapping. This approach is labelled as the cooperative guided mapping approach of PdDE.

Another experimental study is performed on a random selection of the β from the population for mapping. It is observed that when genes of multiple vectors from the population is used it enhances the searching capability of the algorithm. Thus, an empirical analysis is conducted by the implementation of a random selection of second-best

Received: February 4, 2025. Revised: May 5, 2025.

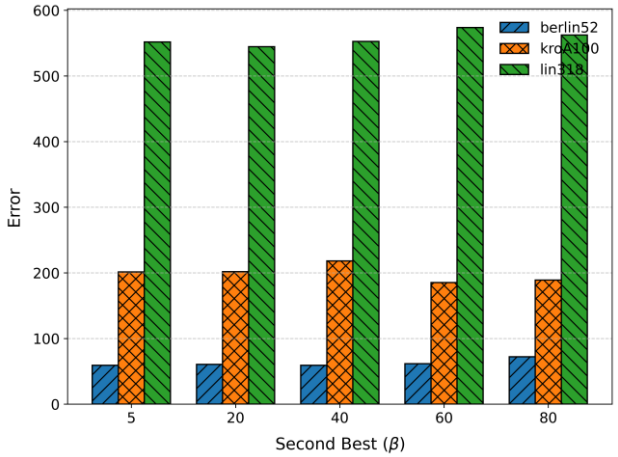

Figure. 2 Average error obtained for TSP instances for different β.

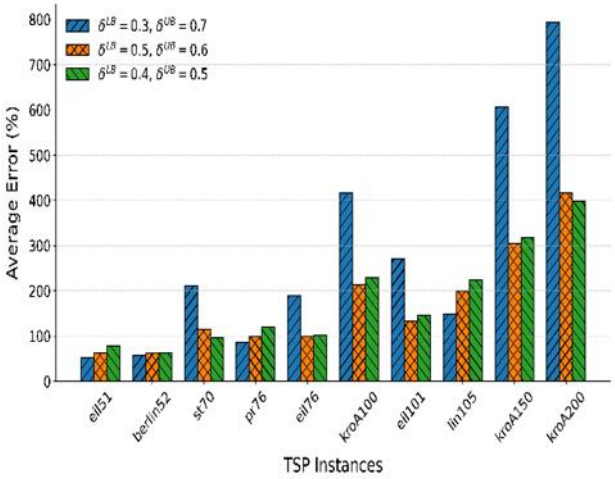

Figure. 3 Average error obtained for TSP instances for different δ^{LB} and δ^{UB}.

individuals from the best-ranked top 5, 20, 40, 60, and 80 individuals. Fig. 2 shows the error E_{rr} obtained for a diverse selection of values of β for the TSP instances - berlin52, kroA100, and lin318.

Analysing the result obtained, β is randomly selected from the top five individuals from the population to balance the exploration of DE. It is observed that for the problem instances with smaller and higher numbers of cities, random selection from less fitness individuals resulted in stagnation. Performing mapping with top-ranked vectors from the population resulted in fast premature convergence.

6.1 Analysis of δ^{LB} and δ^{UB}

A quantitative performance evaluation of PdDE is performed on 10 TSP instances, employing three distinct sets of boundary values denoted as bv_1, bv_2, and bv_3. Table 3 summarizes the average error obtained across these instances for each set of boundary values. The bv_1 demonstrated limited effectiveness, particularly for instances with higher and average number of cities.

Table 3. Average error of PdDE for different δ^{LB} and δ^{UB}

Cities (n)	TSP Instance	Error		
		bv_1	bv_2	bv_3
		δ^{LB} = 0.3 δ^{UB}= 0.7	δ^{LB} = 0.5 δ^{UB}= 0.6	δ^{LB} = 0.4 δ^{UB}= 0.5
51	eil51	51.75	62.47	77.86
52	berlin52	56.54	61.37	62.25
70	st70	210.07	115.25	97.04
76	pr76	85.35	98.66	119.97
76	cil76	188.78	98.94	102.42
100	kroA100	417.19	213.5	228.8
101	cil101	271.07	133.23	145.79
105	lin105	149.54	199.02	224
150	kroA150	606.34	304.25	317.4
200	kroA200	794.33	417.34	398.32
Average		**283.09**	**171.06**	**177.38**

Notably, bv_1 yielded lower error rates for instances with lower number of city nodes such as eil51, berlin52, pr76, and lin105. However, its performance is outperformed by bv_2 and bv_3 for instances st70, eil76, kroA100, eil101, kroA150, and kroA200. This discrepancy suggests that while selecting 30% of genes from both upper and lower bounds facilitated faster exploitation and exploration for small number of cities, it resulted in stagnation as city nodes increased.

Fig. 3 presents the performance differences across various probabilities for the TSP instances. Significant deviations in the results for bv_3 are evident, such as bv_1 exhibiting superior performance for lin105 with a marginal difference of 49.48. Additionally, bv_3 outperforms bv_2 for st70 by a margin of 18.21. Overall, bv_2 demonstrated superior performance compared to both bv_1 and bv_3, with an average error difference of 6.28 and 112.03, respectively.

The decision to further investigate bv_2 is motivated by its ability to select 90% of genes for mapping with the best genes, thereby enhancing the exploration and exploitation capabilities, particularly for instances with more cities. In contrast, for instances with fewer cities, the strategy of mapping fewer genes contributed to enhanced performance. The comparison of average error values across different sets of boundary values emphasizes the crucial role of mapping parameter selection in the algorithmic performance. The bv_2 consistently demonstrated the lowest average error values across

 DOI: 10.22266/ijies2025.0731.09

most of the TSP instances, indicating that the combination of δ^{LB} = 0.5 and δ^{UB} = 0.6 is particularly favourable. Despite performance variations, the average error values remained within reasonable bounds for all configurations, highlighting the efficacy of the proposed approach in addressing TSP instances.

6.2 Performance analysis of sadDE for varying crossover rates

Based on an empirical performance analysis of SadDE with varying values of crossover rates (CR), it is observed that the relationship between the solution quality and the CR values is dataset-dependent. The results recorded for this study are presented in Table 4. No significant performance difference is observed for the instances of eil51, st70, and pr76, while lower values of CR are more effective for the instances eil51, berlin52 eil76 and kroA100. SadDE exhibits varied performance based on the chosen CR for different instances. Generally, it is observed that instances with more number of cities benefit from lower CR values.

Fig. 4 represents the heatmap of TSP instances for different CR values of the SadDE algorithm. The values in the heatmap are directly taken from the dataset and are not normalized. Each cell reflects the actual performance value for the corresponding CR settings.

From the analysis, it is observed that CR values of 0.3 and 0.5 performed better for most of the instances. Specifically, CR = 0.3 showed significant performance improvements for larger instances such

Table 4. The performance of SadDE for different CR values

TSP Instances	CR Values				
	0.1	**0.3**	**0.5**	**0.7**	**0.9**
eil51	27.74	33.8	26.11	30.3	33.1
berlin52	33.85	36.37	40.29	34.1	40.52
st70	74.37	78.22	65.78	82.81	81.48
eil76	56.59	61.17	60.25	63	65.02
pr76	72.83	73.46	71.68	78.48	82.04
kroA100	151.79	161.36	130.59	154.22	192.45
kroB100	144.7	162.72	139.11	165.18	151.28
eil101	105.25	108.11	85.85	108.11	101.11
lin105	118.67	131.27	132.74	161.46	199.01
kroA150	229.08	224.22	281.28	276.88	283.38
kroA200	315.89	320.87	358.18	385.46	384.21
pr226	604.19	599.19	689.59	795.88	755.33
pr264	697.73	673.75	752.74	736.79	744.74
pr299	501	465.48	565.73	627.47	612.99
lin318	484.64	413.97	528.58	598.4	551.84

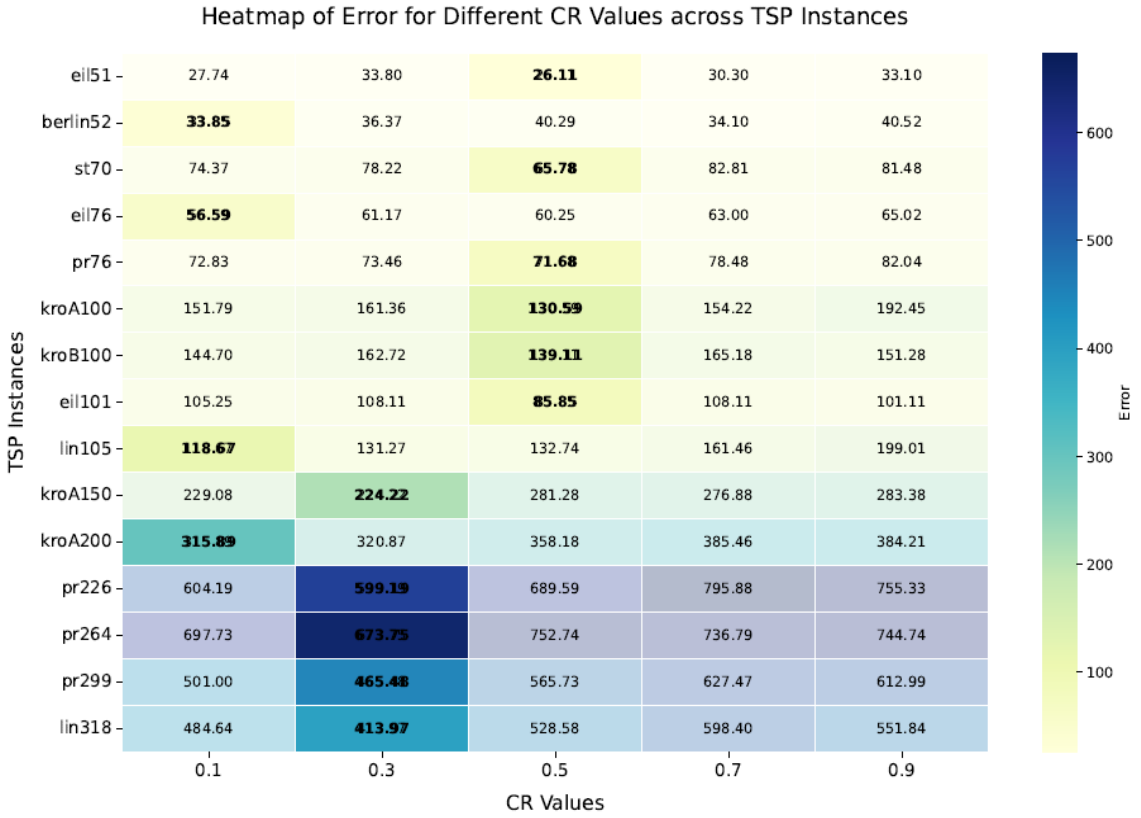

Figure. 4 Heatmap showing the performance of SadDE for different CR values

as pr226, pr264, pr299, and lin318. However, there is no single CR value that is optimal across all instances. The optimal CR value varies depending on the dataset characteristics and problem size. Therefore, experimentation is essential to identify the most suitable CR value for each dataset. The performance of SadDE on the Travelling Salesman Problem (TSP) and 0/1 Knapsack Problem (KP) instances is notably influenced by the choice of CR. Selecting an appropriate CR value is crucial for achieving optimal results, and this optimal value differs across datasets. Although CR = 0.3 yielded better results in our experiments, CR = 0.9 is selected for this study to ensure a fair comparison with state-of-the-art DE mapping variants, as this value aligns with the parameter setting used in previous empirical studies as mentioned in [35].

6.3 Exploration and exploitation analysis of PdDE and SadDE

Further analysis is conducted to gain insights into the exploration and exploitation behavior of the proposed discrete versions of classical DE, the PdDE and SadDE frameworks. It is observed that when gene mapping is performed cooperatively from similar after a few iterations, hindering further multiple vectors, the searching ability of discrete DE versions is increased. But the predetermined δ^{LB} and δ^{UB} lead to premature convergence. Individuals look exploration and exploitation. As iteration proceeds, reducing the size of genes mapped from $\propto$ and β, and introducing mechanisms to maintain diversity could improve the exploration ability of PdDE. This is resolved in SadDE by randomizing the values of δ^{LB} and δ^{UB} as mentioned in equations Eq. (12) and Eq. (13).

The convergence patterns observed at different iterations provide better insight into the exploration and exploitation ability of PdDE and SadDE. The observations are recorded as graphs and are presented in Fig. 5. The self-adaptive mapping approach adjusts gene mapping adaptively based on the optimization process. This approach maintains randomness through probability factors θ_1 and θ_2, promoting exploration even in later iterations. The analysis showed a linear increment in searching with the self-adaptive mapping approach. The adaptability and maintained randomness of SadDE contribute to a more consistent exploration behavior compared to PdDE. In the early iterations, SadDE exhibits

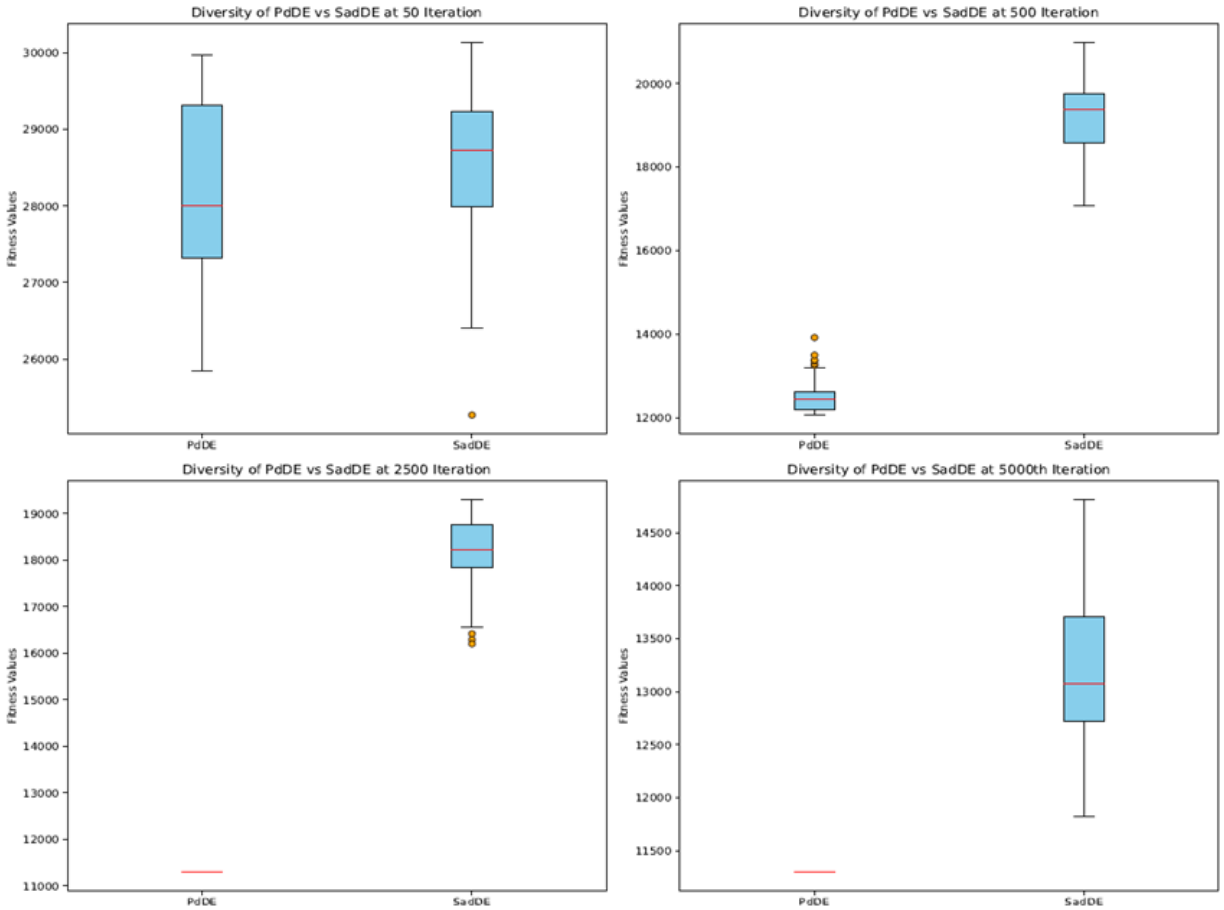

Figure. 5 Convergence analysis of PdDE and SadDE.

diversity in its population, while PdDE rapidly converges to the optimal solution. This approach maintains randomness through probability factors θ_1 and θ_2, promoting exploration even in later iterations. The analysis showed a linear increment in searching with the self-adaptive mapping approach. The adaptability and maintained randomness of SadDE contribute to a more consistent exploration behavior compared to PdDE. In the early iterations, SadDE exhibits diversity in its population, while PdDE rapidly converges to the optimal solution. As the iterations progress, the population diversity decreases for PdDE, with all individuals becoming identical by the 500^{th}iteration.

6.4 Comparison with state-of-the-art DE mapping algorithms

The performance of the proposed framework SadDE is assessed with PdDE and six state-of-the-art mapping algorithms. The algorithms are Truncation Procedure (TP) [22], TP with rounding to the nearest integer number (TPL) [35], TP considering only the integer part (TPI) [35], Rank-based (RB) [45], Largest ranked value (LRV) [46], and Best matched value (BMV) [35]. To compare the performance of the proposed SadDE and PdDE frameworks, the parameter settings and mutation strategy outlined in [35] are considered.

6.4.1. Performance analysis of PdDE on TSP

The performance of PdDE on 15 TSP instances is assessed based on the average error for the best solution against the state-of-the-art mapping approaches for the DE algorithm. The average error is the average of the error percentages of the best solutions obtained for each TSP instance by the algorithms compared to the known optimal solution. The best solutions are selected from the results of 30 independent runs. The results are summarized in Table 5. The average error obtained by PdDE is better than the conventional mapping approaches for all the instances of TSPs. On analysing the average error obtained, it is found that the PdDE outperformed TP with a difference of 349.54, TPL with a difference of 362.16, TPI with a difference of 296.21, outperformed RB with the difference of 337.93, LRV with the difference of 338.70, and BMV by obtaining less error with a difference of 96.72. Based on the empirical analysis on the performance of the mapping approaches they are ranked in the order of PdDE, BMV, TPI, RB, LRV, TP, and TPL.

6.4.2. Performance analysis of SadDE on TSP

As shown in Table 5, the average error values of the TP, TPL, TPI, RB, BMV, PdDE and SadDE and LRV are 671, 683.62, 617.67, 659.39, 660.16, 418.18, 321.46, and 285.23, respectively. The SadDE framework has secured the lowest average error value among all the mapping approaches. The percentage of improvements (*PI*) attained by the SadDE in comparison with each of the counterpart mapping approaches is calculated using the equation (17).

$$PI = \frac{AE_C - AE_{SadDE}}{AE_C} \quad (17)$$

where AE_C is the average error of the counterpart mapping approach, and AE_SadDE is the average error of SadDE.

Table 5. Comparison of PaDE and SadDE with the state-of-the-art DE mapping approaches for TSP instances

Cities (n)	Instance	TP	TPL	TPI	RB	LRV	BMV	PdDE	SadDE
50	eli51	177.64	186.06	182.12	188.93	190.85	78.57	62.47	**33.51**
52	berlin52	179.35	179.33	180.03	196.65	179.58	78.56	61.41	**40.52**
70	st70	308.1	288.89	313.55	317.57	309.86	142.92	115.25	**81.48**
76	pr76	257.97	260.34	260.12	273.78	263.55	123.83	98.65	**82.04**
96	eil76	288.83	295.18	288.3	291.48	311.58	145.94	98.94	**65.02**
100	kroA100	504.46	501.37	517.26	526.4	545.67	268.44	213.49	**192.45**
100	kroB100	469.38	451.84	448.07	454.92	450.07	262.44	183.96	**151.28**
101	eil101	331.33	328.19	320.66	339.88	340.8	173.85	133.22	**101.11**
105	lin105	503.57	544.57	552.27	466.78	609.37	302.15	199.02	**199.01**
150	kroA150	631.8	651.58	687.53	696.45	497.32	459.04	304.25	**283.38**
200	kroA200	850.13	859.81	850.45	856.51	896.41	672.03	417.33	**384.21**
226	pr226	1559.91	1611.88	1145.84	1273.52	1673.67	871.44	811.86	**755.33**
264	pr264	1669.05	1705.82	1163.18	1714.38	1214.29	821.29	845.45	**744.74**
299	pr299	1232.4	1256.15	1240.77	1208.26	1250.99	989.18	650.81	**612.99**
318	lin318	1113.42	1132.94	1115.18	1085.37	1168.39	883.12	625.99	**551.84**
Average		671	683.62	617.67	659.39	660.16	418.18	321.46	**285.23**

The measured performance improvement percentages are 57.49% with TP, 58.28% with TPL. 53.82% with TP, 56.74% with RB, 56.79% with LRV, 31.79% with BMV, and 11.27% with PdDE. Fig. 6 shows the PI values of SadDE in comparison with other DE mapping approaches.

The convergence of SadDE and PdDE with existing algorithms for the TSP instances of berlin52, kroA100, kroA200, and lin318 is shown in Fig. 7.

On analysing the convergence of TP, TPL, TPI, RB, and LRV, it is noted that the solution quality improves slowly. Better convergence can be observed for BMV when compared with TP, TPL, TPI, RB and LRV. PdDE exhibited rapid convergence during the initial iterations, leading to better solutions compared to other DE variants at the early stages. However, as the iterations progressed, premature convergence is observed. In contrast, SadDE demonstrated consistent improvement in solution quality throughout the iterations, indicating the algorithm's effective progression towards better results.

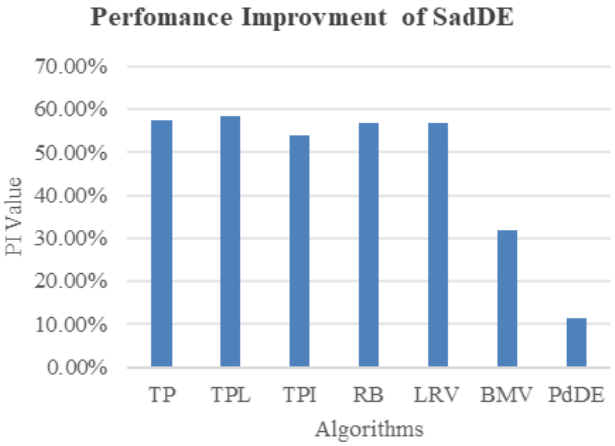

Figure. 6 Performance improvement of SadDE with other algorithms.

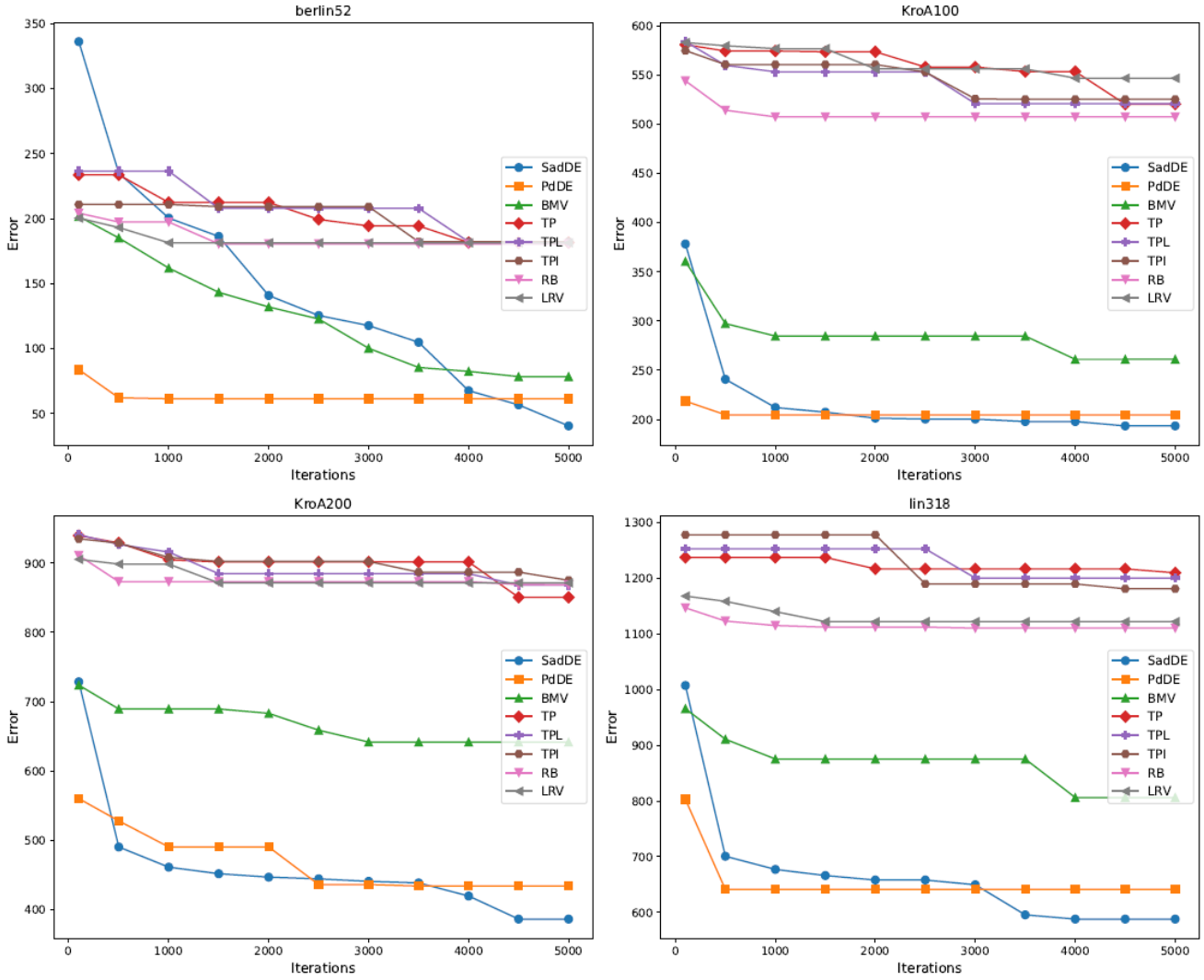

Figure. 7 Convergence analysis of PdDE, SadDE, and other DE mapping approaches for berlin52, kroA100, kroA200, and lin318

6.4.3. Integration and validation of PdDE and SadDE with state-of-the-art approaches

To validate the performance of the proposed PdDE and SadDE approaches with state-of-the-art, the experiments are carried out in two phases. In Phase I, the PdDE and SadDE are integrated with the popular DE mapping approaches of TP, TPL, TPI, RB, LRV, and BMV. In Phase 2, the BCO, GA, FA, and GWO algorithms are incorporated with the PdDE and SadDE. In both phases, the classical and integrated versions of the algorithms are compared.

Table 6 presents the error percentage of the best solutions of the classical DE mapping approaches and their corresponding PdDE and SadDE integrated versions, for the TSP instances berlin52, kroA200, and lin318. It is observed from the results that the PdDE and SadDE versions are outperforming their counterpart classical approaches with lower error values, except for the case of RB_PdDE in kroA200. In the case of kroA200, the RB_PdDE failed to outperform RB. However, RB_SadDE significantly outperformed RB in this case. It is also interesting to note that the SadDE versions have outperformed the corresponding PdDE in all the cases.

Table 7 represents the comparison of DE existing mapping approaches integrated with SadDE and PdDE with higher-dimensional TSP instances with a large number of cities – the rat575 and pr1002. This study is done to test the scalability of SadDE. Table 7 also presents the error percentage of the best solutions. The result shows that DE mapping variants integrated with SadDE showed consistently better performance than the corresponding PdDE versions, for both TSP instances. Overall, integration of PdDE and SadDE with various discrete DE mapping strategies resulted in better performance than classical mapping methods for TSP.

Table 6. Comparison of state-of-the-art DE mapping approaches combined with PdDE and SadDE for TSP

	berlin52	kroA200	lin318
TP	179.35	850.13	1113.42
TP_PdDE	75.78	595.24	945.71
TP_SadDE	15.13	434.52	587.09
TPL	179.33	859.81	1132.94
TPL_PdDE	75.07	589.9	621.35
TPL_SadDE	15.91	414.87	583.92
TPI	180.03	850.45	1115.18
TPI_PdDE	76.16	577.89	588.5
TPI_SadDE	17	425.98	567.53
RB	196.65	856.51	1085.37
RB_PdDE	71.68	1041.12	697.36
RB_SadDE	59.84	395.59	568.99
LRV	179.58	896.41	1168.39
LRV_PdDE	50.12	585.94	848.94
TPL_SadDE	25.27	272.24	652.74
BMV	78.56	672.03	883.12
BMV_PdDE	67.13	490.86	658.55
BMV_SadDE	62.74	366.57	550.46

Table 7. Comparison of PdDE and SadDE for higher dimensional TSP instances

	rat575	pr1002
TP_PdDE	1277.25	1555.9
TP_SadDE	932.67	1393.67
TPL_PdDE	935.69	1419.33
TPL_SadDE	902.92	1366.96
TPI_PdDE	909.7	1433.87
TPI_SadDE	870.04	1424.16
RB_PdDE	1046.61	1760.45
RB_SadDE	848.91	1500.22
LRV_PdDE	1162.68	2139.23
LRV_SadDE	1118.23	1912.8
BMV_PdDE	981.04	1751.7
BMV_SadDE	755.09	1469.81

PdDE and SadDE have significantly improved the performance of conventional standalone DE mapping strategies by enhancing both solution quality and convergence behavior. Among the two, SadDE consistently outperformed PdDE due to its self-adaptiveness, which ensures a good balance between exploration and exploitation. As a sample, the convergence graph of the SadDE for rat575 is shown in Fig. 8.

However, despite its superior performance, the SadDE showed early convergence in some instances and computational overhead. SadDE is further investigated on classical metaheuristic algorithms, BCO, GA, FA, and GWO. Comparative analysis of classical metaheuristics (BCO, GA, FA, and GWO) and their integration with SadDE is conducted. The parameter setting for the experiment is considered as mentioned in Table 8.

The metrics used for the comparison are the best error value and the average error value. In this experimental part, two higher-dimensional TSP instances are added to test the scalability of SadDE. They are the rat575 with 575 cities and pr1002 with 1002 cities. Results are shown in Tables 9 and 10.

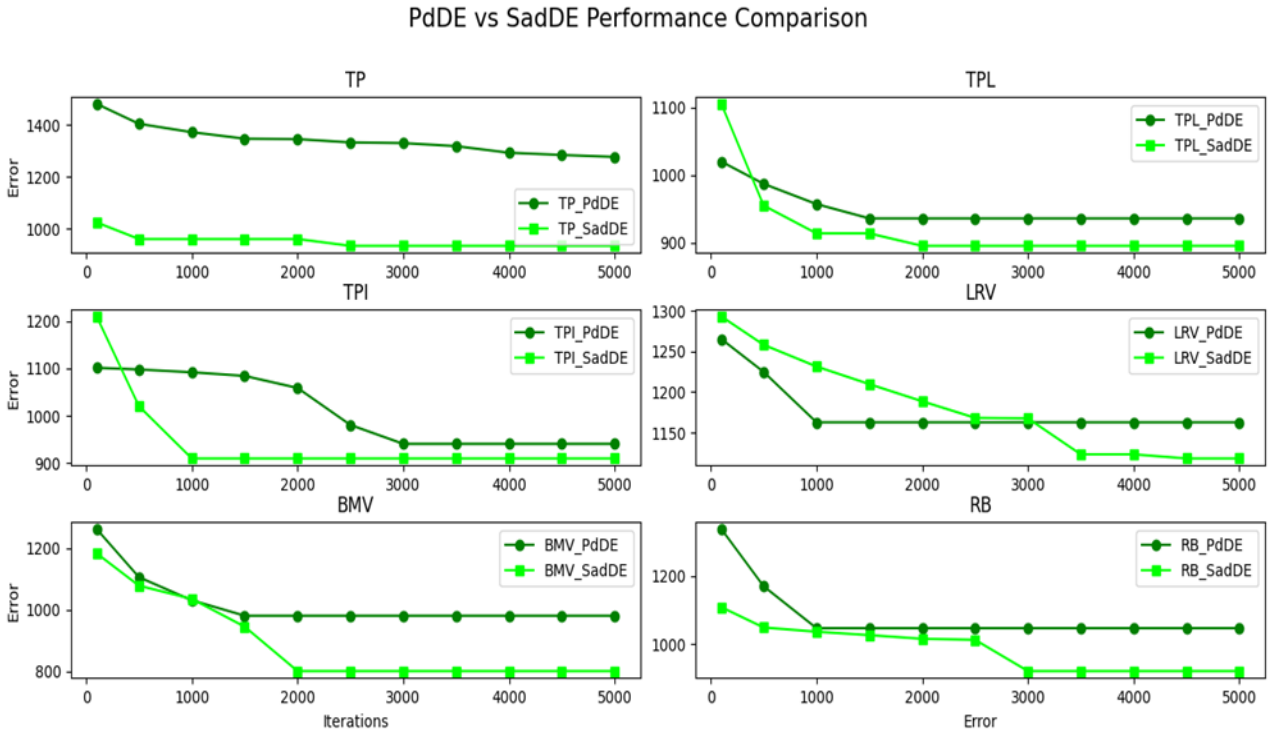

Figure. 8 Convergence analysis of the existing DE mapping approach integrated with PdDE and SadDE for rat575.

Table 8. Parameter settings for metaheuristic algorithms

BCO	Pop_size=100, employed_bee=50, onlooker_bee=50, iteration = 1000, scout bees = 5, mutation = swap, selection = fitness-Proportional
GA	Pop_size=100, iteration = 1000, elite size = 1, crossover = order crossover, mutation rate = 0.1, selection = rank based
FA	Pop_size=100, iteration = 1000, alpha = 0.5, alpha_decay = 0.99, beta = 1.0, gamma = 1.0, mutation = swap
GWO	Pop_size=100, iteration = 1000, position update= swap, mutation = swap, selection = best of population

The error percentages of the best solution (denoted as "Best") and the average solution (denoted as "Avg") are presented in these tables. From Table 9, it is observed that the BCO combined with SadDE improved the quality of the solution. The average error is reduced from 358.22 to 257.75, and the error for the best solution decreased from 333.49 to 233.66. Similarly, the GA integrated with SadDE also resulted in an improvement in the solution quality for both the average and best solutions.

Table 10 also shows a similar trend for FA and GWO. The FA with SadDE reduced the average error for the best solution from 775.09 to 491.58. The

Table 9. Comparison of classical BCO and GA with BCO_SadDE, and GA_SadDE for TSP

TSP Instances	BCO		BCO_SadDE		GA		GA_SadDE	
	Best	Avg	Best	Avg	Best	Avg	Best	Avg
eil51	19.72	31.82	16.9	33.98	82.16	99.48	25.35	43.65
berlin52	21.64	35.63	18.43	36.89	76.09	94.01	37.22	51.37
kroA100	120.37	155.13	77.49	101.37	286.45	326.72	152.22	181
eil101	84.1	92.68	46.26	61.46	199.05	218.04	102.86	121.48
lin105	124.72	160.2	83.36	113.22	315.72	361.81	171.72	214.92
kroA150	197.08	239.74	119.03	144.81	429.19	470.52	257.34	286.73
rat575	737.71	753.25	489.10	524.04	1017.82	1060.8	750.2	812.54
pr1002	1362.58	1397.36	1018.71	1046.23	1693.75	1730.82	1449.28	1498.77
Average	333.49	358.226	233.66	257.75	512.529	545.275	368.27375	401.3075

Table 10. Comparison classical FA and GWO for GWO_SadDE and FA_SadDE for TSP

TSP Instances	FA		FA_SadDE		GWO		GWO_SadDE	
	Best	Avg	Best	Avg	Best	Avg	Best	Avg
eil51	182.39	191.62	72.07	90.87	118.08	148.46	96.71	115.87
berlin52	183.13	197.07	67.5	89.55	113.76	198.09	83.59	115.48
kroA100	521.91	539.64	219.31	270.16	386.93	439.28	292.74	355.97
eil101	336.16	346.32	162.13	191.13	250.24	341.18	188.39	220.52
lin105	569.64	585.1	268.06	299.88	389.28	485.18	322.89	367.49
kroA150	707.57	716.55	352.01	383.18	500	592.78	410.77	471.17
rat575	1455.41	1613.62	1006.53	1030.42	1321.9	1372.51	1135.13	1173.6
pr1002	2244.56	2279.29	1785.07	1807.31	2172.8	2241.41	1903.02	1952.6
Average	775.09	808.65	491.59	520.31	656.62	727.36	554.15	596.59

GWO integrated with SadDE also showed a decrease in error rate for the average solution from 727.36 to 596.59, and in the best solution from 656.62 to 554.15. Combining these observations, the performance improvement percentage of the SadDE integrated metaheuristic algorithms by the 'Best" values are 28.05%, 26.40%, 35.66%, and 19.98%, respectively for the classical BCO, GA, FA, and GWO algorithms. In the same order, the performance improvement percentage by the 'Avg' values are 29.93%, 28.15%, 32.87%, and 9.14%.

Two exceptional cases are found in this comparison (Refer to Table 9). The BCO and BCO_SadDE comparison for eil51 and berlin52 shows that the BCO outperforms BCO_SadDE marginally on the error percentage of the average solutions.

An exclusiveve analysis on SadDE integrated algorithms on the higher-dimensional problem instances (rat575 and pr1002) is carried out, and the performance improvement percentage values are presented in Table 11. The results show the superiority of the proposed SadDE approach in higher-dimensional problem instances.

Taking one lower dimension TSP instance (berlin52) and one higher dimension TSP instance (pr1002), the convergence nature of the classical metaheuristics and their SadDE versions are presented in Figs. 9 and 10.

Table 11. Performance improvement of SadDE on high-dimensional problem instances

TSP Instance	BCO_SadDE's PI with BCO		GA_SadDE's PI with GA	
	Best	Avg	Best	Avg
rat575	33.70%	30.43%	35.67%	30.55%
pr1002	25.24%	25.13%	16.87%	15.48%
TSP Instance	FA_SadDE's PI with FA		GWO_SadDE's PI with GWO	
	Best	Avg	Best	Avg
rat575	30.84%	36.14%	14.13%	14.49%
pr1002	20.47%	20.71%	12.42%	12.89%

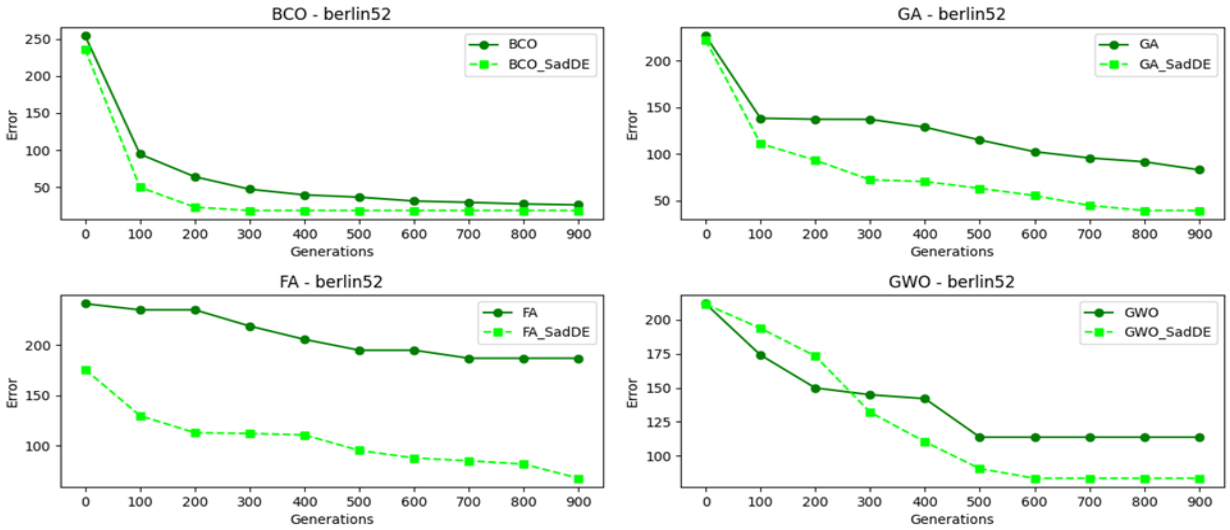

Figure. 9 Convergence analysis of classical BCO, GA, FA, and GWO and their SadDE version for berlin52.

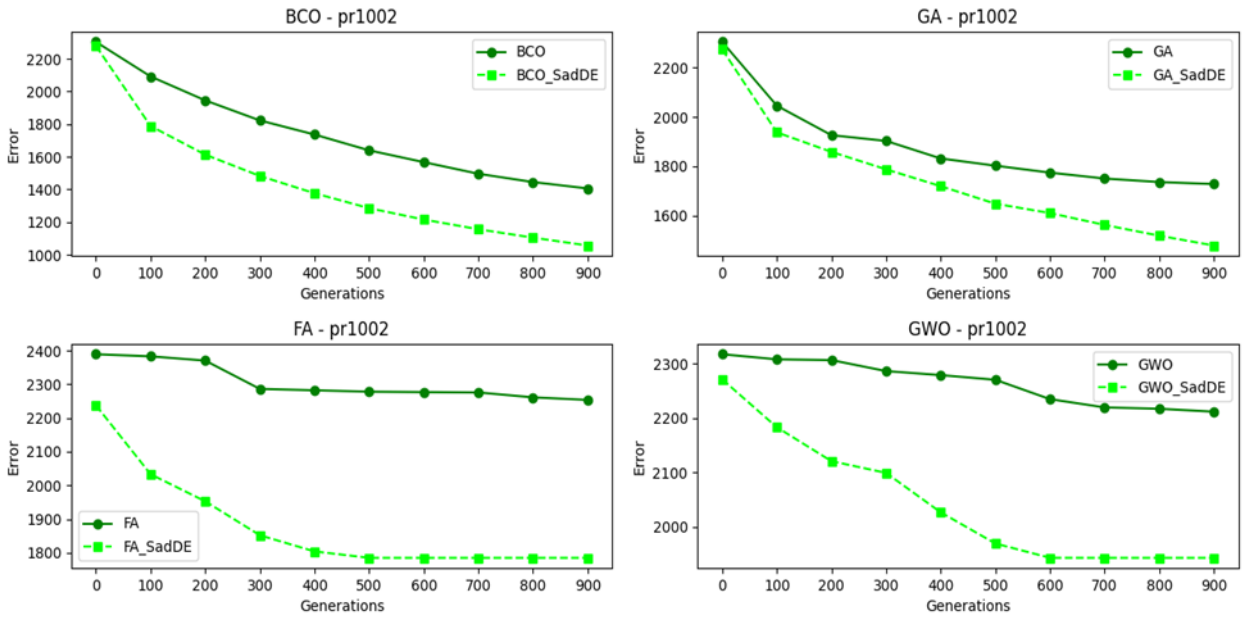

Figure. 10 Convergence analysis of classical BCO, GA, FA, and GWO, and their SadDE version for pr1002.

6.4.4. Analysis of PdDE on 0/1 KP

For solving the 0/1 KP problem instances, the performance of the PdDE is appraised against other mapping approaches. Table 12 shows the results achieved for eight instances of 0/1 KP, denoted as P01, P02, P03, P04, P05, P06, P07, and P08. Each instance is characterized by the total number of items and the optimal solution value. The error obtained for each algorithm (TP, TPL, TPI, RB, LRV, BMV, PdDE, and SadDE) is calculated as the percentage difference between the algorithm's solution and the optimal solution. An error of 0 indicates that the algorithm found the optimal solution. For instances P011, P02, P03, P04, and P06, all the algorithms achieved an error of 0, indicating that they found the optimal solution.

The PdDE outperformed other approaches, significantly, for the instances P05, P07 and P08. The PdDE approach achieved an optimal solution for P05, a 99.99% result for P07, and a 99.58% result for P08. The PdDE outperformed the approaches of TP, TPL, TPI, RB, and LRV for the problem instance P07 with a difference of value 89.963.

The PdDE and outperformed BMV with a marginal difference of 0.003 for P07. For the problem instance P08, the PdDE approach outperformed BMV and RB with a difference of 0.72, outperformed TP with a difference of 1.39, TPL with a difference of 0.91, TPI with a difference of 1.62, and the LRV with a difference of 1.41. Based on the empirical analysis of the obtained results, the mapping approaches are ranked in the order of PdDE, *BMV*, *RB*, *TPL*, *TP*, *LRV*, and *TPI*. In the cooperative guided mapping approach of the PdDE, gene mapping of mutant vectors and the selected individuals is performed.

Table 12. Comparison of SadDE and PdDE with state-of- the-art algorithms for 0/1-KPs

0/1-KPs	Total items	Optimal solution	Error							
			TP	TPL	TPI	RB	LRV	BMV	PdDE	SadDE
P01	10	309	0	0	0	0	0	0	0	0
P02	5	51	0	0	0	0	0	0	0	0
P03	6	150	0	0	0	0	0	0	0	0
P04	7	107	0	0	0	0	0	0	0	0
P05	8	900	1.12	1.12	1.12	1.12	1.12	1.12	0	0
P06	7	1735	0	0	0	0	0	0	0	0
P07	15	1458	90.1	90.1	90.1	90.1	90.1	0.14	0.137	0.06
P08	24	13496748	1.81	1.33	2.04	1.14	1.83	1.14	0.42	0.41
Average			**11.62**	**11.56**	**11.65**	**11.54**	**11.63**	**0.3**	**0.069**	**0.06**

The mutant vectors are converted to discrete by ranking their genes. The ranked mutant vectors are undergoing the proposed mapping approach. The conversion of discrete values of the mutant to binary is performed based on [35].

6.4.5. Analysis of SadDE on 0/1 KP

For evaluating the performance of the SadDE algorithm in solving the 0/1 Knapsack Problem (0/1 KP) instances, it is compared with state-of-the-art mapping approaches. Table 12. presents the comparison of the results.

- For the problem instance P05, the SadDE and PdDE achieved an error percentage of 0, indicating that they obtained an optimal solution. In comparison with other mapping approaches, including BMV, there is an error difference of 1.12%. This indicates that SadDE and PdDE outperform TP, TPL, TPI, RB, LRV, and BMV, significantly.
- For the problem instance P07, the SadDE obtained an error of 0.06, which is the lowest among all the mapping approaches. In comparison, the other approaches TP, TPL, TPI, RB, LRV, and BMV incurred errors of 90.1, 90.1, 90.1, 90.1, 90.1, and 0.14, respectively. The PdDE obtained an error of 0.137. This indicates that SadDE significantly outperformed TP, TPL, TPI, RB, LRV, and BMV. It performed better than PdDE, marginally. This reiterates the superior performance of SadDE.
- The SadDE had an error of 0.41 for the problem instance P08, which is the lowest among all the approaches. In comparison, other algorithms such as TP, TPL, TPI, RB, LRV, and BMV incurred errors ranging from 1.14 to 2.04. The PdDE obtained an error of 0.42. This indicates that SadDE performed significantly better than TP, TPL, TPI, RB, LRV, and BMV, and marginally better than PdDE.

On average, SadDE had an error of 0.060, which is the lowest among other approaches. The TP, TPL, TPI, RB, LRV, and BMV approaches incurring errors ranging from 11.54 to 11.65. The PdDE achieved an error of 0.069. This indicates that SadDE outperformed TP, TPL, TPI, RB, LRV, and BMV, and performed slightly better than PdDE on average.

Table 13 presents the average computational time (ACT) obtained for TSP instances and 0/1 KP instances. For the TSP instances, the ACT value is lesser for the BMV and TP approaches. For the 0/1 KP instances, the RB approach takes a lesser ACT value.

6.4.6. Statistical analysis

To validate the results obtained, statistical significance analysis is performed using Wilcoxon Signed Ranks Test for Paired Samples with two tails for the comparison of SadDE with other algorithms for TSP and 0/1 KP datasets at 95% confidence level. Three signs +, = and – are used to represent the significance. The '+' represents that the first algorithm is significantly better. The '=' represents that there is no significant difference and the '-' indicates that the first algorithm performed significantly worst. From the statistical analysis shown in Table 14, it is apparent that the self-adaptive mapping approach of SadDE significantly outperformed state-of-the-art mapping approaches for all the instances of TSP. For 0/1 KP, the self-adaptive mapping approach of SadDE, significantly outperformed the state-of-the-art mapping approaches for the problem instances P05, P07, and P08 and showed equal performance for P01, P02, P03, P04, and P06.

Table 13. Average computation time comparison

Algorithm	ACT for TSP(s)	ACT for 0/1 KP (s)
SadDE	1176.63	86.05
PdDE	1162.74	109.265
BMV	1102.45	68.26
TP	1102.66	59.86
TPL	1131.23	56.25
TPI	1142.11	58.5
RB	1163.51	53.6
LRV	1151.90	55.9

Table 14. Statistical Analysis

TSP					
Methods	**>**	**=**	**<**	**pval**	**Significance**
SadDE vs PdDE	15	0	0	0.0043	+
SadDE vs BMV	15	0	0	0.0042	+
SadDE vs TP	15	0	0	0.0029	+
SadDE vs TPL	15	0	0	0.003	+
SadDE vs TPI	15	0	0	0.0024	+
SadDE vs RB	15	0	0	0.0028	+
SadDE vs LRV	15	0	0	0.0039	+
0/1 KP					
Methods	**>**	**=**	**<**	**pval**	**Significance**
SadDE vs PdDE	2	6	0	0.0015	+
SadDE vs BMV	3	5	0	0.0015	+
SadDE vs TP	3	5	0	0.0012	+
SadDE vs TPL	3	5	0	0.002	+
SadDE vs TPI	3	5	0	0.002	+
SadDE vs RB	3	5	0	0.0017	+
SadDE vs LRV	3	5	0	0.0016	+

7. Conclusion

The primary focus of the work presented in this paper is to propose a novel mapping strategy for DE to solve combinatorial optimization problems. The recent studies in the literature are focusing mainly on operator adaptations or the hybridization of DE with other metaheuristics for designing DE for COPs. The studies also suggest that the mapping strategies need mechanisms to handle the infeasible solutions and to balance the exploration and exploitation with lesser computational time.

In this paper, two novel DE frameworks (named PdDE and SadDE) are proposed for solving COPs. The PdDE and the SadDE are incorporated with two proposed mapping approaches named the 'cooperative guided mapping approach' and the 'self-adaptive mapping approach', respectively.

The proposed frameworks are validated in two sets of benchmark COP instances – the TSP and the 0/1 KP. During the investigation of the performance of PdDE, it is observed that as generation increases, it proceeds toward the optimal solution faster and falls in premature convergence. To overcome this issue the SadDE framework is designed. In SadDE, a self-adaptive mapping strategy is incorporated with the DE algorithm where genes of the multiple best vectors are mapped adaptively to the mutant vector and a few selected vectors, in the current population. The SadDE could direct DE towards the optimal solution faster. During the investigation, it is observed that the mapping of best genes from multiple best individuals improved the performance of DE for solving the problem instances of TSP with The SadDE algorithm enhanced the searchability of DE adaptively by ensuring that the vectors produced are new. The exploration and exploitation are thus balanced by SadDE. Experimental findings demonstrated that SadDE's competency significantly outperformed the extant mapping reported in the literature by obtaining promising results. The SadDE could competitively perform well for permutation-based discrete optimization problems. The SadDE's prime limitation is its computational time, as it maps the gene values with multiple individuals. It is also observed that number of cities also affecting the exploitation ability of DE over the increase in generation. The future work of this study is to focus on the reducing the computation time issue of the SadDE. This work can be extended with a strategy to improve the exploitation of algorithm for larger instances.

Conflicts of Interest

The authors declare no conflict of interest.

Author Contributions

Conceptualization, Methodology: Anisha Radhakrishnan and G Jeyakumar, Data curation, investigation, Writing: Anisha Radhakrishnan. Validation, Supervision, Review, and Editing: G Jeyakumar.

References

[1] R. Storn, and K. Price, "Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces", *Journal of Global Optimization*, Vol. 11, No. 341-359, 1997.

[2] M. Salomon, G. R. Perrin, F. Heitz, and J. P. Armspach, "Parallel differential evolution: application to 3-D medical image registration", *Differential Evolution: A Practical Approach to Global Optimization*, pp. 353-411, 2005.

[3] J. Adeyemo, F. Bux, and F. Otieno, "Differential evolution algorithm for crop planning: Single and multi-objective optimization model", *International Journal of Physical Sciences*, Vol. 5, No. 10, 1592-1599, 2010.

[4] A. T. Buba, and L. S. Lee, "A differential evolution for simultaneous transit network design and frequency setting problem", *Expert Systems with Applications*, Vol. 106, No. 277-289, 2018.

[5] S. Harini, K. Mamaikya, V. Chandramouli, S. Sundar, and S. Natarajamani, "Compact Planar MIMO Antenna Isolation Structure Based on Evolutionary Algorithm", In: *Proc. of Second International Conf. on Electronics and Sustainable Communication Systems (ICESC)*, Coimbatore, India, pp - 663-666, 2021

[6] E. B. Machin, D. T. Pedroso, A. B. Machin, D. G. Acosta, MISd. Santos, FSd. Carvalho, N. P. Perez, R. Pascual. and JAdC. Juniot, "Biomass integrated gasification-gas turbine combined cycle (BIG/GTCC) implementation in the Brazilian sugarcane industry: Economic and environmental appraisal", *Renewable Energy* Vol. 172, pp. 529-540, 2021.

[7] J. N. Sahu, R. R. Karri, and N. S. Jayakumar, "Improvement in phenol adsorption capacity on eco-friendly biosorbent derived from iste Palm-oil shells using optimized parametric modelling of isotherms and kinetics by differential evolution", *Industrial Crops and Products*, Vol. 164, No. 113333, 2021.

[8] M. Ritwik, and C. S. Velayutham, "A preliminary investigation into automatically evolving computer viruses using evolutionary

algorithms", *Journal of Intelligent & Fuzzy Systems*, Vol. 38, No. 5, pp. 6517-6526, 2020.

[9] S. Vaishnavi, M. N. Devi, and M. Jayakumar, "Data augmented hardware trojan detection using label spreading algorithm based transductive learning for edge computing-assisted IoT devices", *IEEE Access*, Vol. 10, pp. 102789-102803, 2022.

[10] K. Vikram, H. P. Menon, and D. M. Dhanalakshmy. "Segmentation of brain parts from MRI image slices using genetic algorithm", *Computational Vision and Bio Inspired Computing, Lecture Notes in Computational Vision and Biomechanics*, Vol. 28, 2018.

[11] G. Liu, Y. Li, L. Jiao, Y. Chen. and R. Shang, "Multiobjective evolutionary algorithm assisted stacked autoencoder for PolSAR image classification", *Swarm and Evolutionary Computation*, Vol. 60, No. 100794, 2021.

[12] E. Hancer, B. Xue, and M. Zhang, "Differential evolution for filter feature selection based on information theory and feature ranking", *Knowledge-Based Systems*, Vol. 140, pp. 103-119, 2018.

[13] T. S. Rao, "A comparative evaluation of GA and SA TSP in a supply chain network", *Materials Today: Proceedings*, Vol. 4, No. 2, pp. 2263-2268, 2017.

[14] M. A. Elaziz, S. Xiong, K. P. N. Jayasena, and L. Li, "Task scheduling in cloud computing based on hybrid moth search algorithm and differential evolution", *Knowledge-Based Systems*, Vol. 169, No. 1, pp. 39-52, 2019.

[15] H. Mühlenbein, M. G. Schleuter, and O. Krämer, "Evolution algorithms in combinatorial optimization", *Parallel computing*, Vol. 7, No. 1, pp. 65-85, 1988.

[16] P.E.O, Yumbla, J. M. Ramirez, and C. A. C, Coello, "Optimal power flow subject to security constraints solved with a particle swarm optimizer", *IEEE Transactions on Power Systems*, Vol. 23, No. 1, pp. 33-40, 2008.

[17] G. Venter, and J. S. Sobieski, "Multidisciplinary optimization of a transport aircraft wing using particle swarm optimization", *Structural and Multidisciplinary Optimization*, Vol. 26, pp. 121-131, 2004.

[18] R. P. Parouha, andd K.N. Das, "A robust memory based hybrid differential evolution for continuous optimization problem", *Knowledge-Based Systems*, Vol. 103, pp. 118-131, 2016.

[19] S. Das, and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art", *IEEE Transactions on Evolutionary Computation*, Vol. 15, No. 1, pp. 4-31, 2010.

[20] R. S. Prado, R.C.P. Silva, F.G. Guimaraes, and O.M. Neto, "Using differential evolution for combinatorial optimization: A general approach", In: *Proc. of IEEE International Conf. on Systems, Man and Cybernetics*, Istanbul, Turkey, 2010.

[21] A. L. Maravilha, J.A. Ramirez, and F. Campelo, "A new algorithm based on differential evolution for combinatorial optimization", In: *Proc. of 2013 BRICS Congress on Computational Intelligence and 11th Brazilian Congress on Computational Intelligence*, Ipojuca, Brazil, 2013.

[22] R. Anisha, and G. Jeyakumar, "Evolutionary algorithm for solving combinatorial optimization—A review", In: *Proc. of 8th International Conf.* on *Innovations in Computer Science and Engineering* (*ICICSE* 2021), Vol. 171, pp. 539-545, 2021.

[23] T.B. Beielstein, and M. Zaefferer, "Model-based methods for continuous and discrete global optimization", *Applied Soft Computing*, Vol. 55, pp. 154-167, 2017.

[24] J. Kennedy, and R. C. Eberhart, "A discrete binary version of the particle swarm algorithm", In: *Proc. of IEEE International Conf. on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation*, Orlanda, USA, Vol. *5*, pp. 4104-4108, 1997.

[25] J. Lampinen, and I. Zelinka, "Mixed integer-discrete-continuous optimization by differential evolution", In: *Proc. of the 5th International Conf. on Soft Computing*. Vol. 71, 1999.

[26] G. Onwubolu, and D. Davendra, "Scheduling flow shops using differential evolution algorithm", *European Journal of Operational Research*, Vol. 171, No. 2, pp. 674-692, 2006.

[27] L.V. Snyder, and M.S. Daskin, "A random-key genetic algorithm for the generalized travelling salesman problem", *European Journal of Operational research*, Vol. 174, No. 1, pp. 38-53, 2006.

[28] M.F. Tasgetiren, Y.C. Liang, M. Sevkli, and G. Gencyilmaz, "A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem", *European journal of operational research*, Vol. 177, No. 3, pp. 1930-1947, 2007.

[29] D. Lichtblau, "Relative position indexing approach", *Differential evolution: a handbook for global permutation-based combinatorial optimization, Studies in Computational Intelligence*, Berlin, Heidelberg: Springer Berlin Heidelberg, Vol. 175, pp. 81-120, 2009.

[30] F. Tasgetiren, A. Chen, G. Gecyilmaz, and S. Gattoufi, "Smallest position value approach", *Differential Evolution: a Handbook for Global Permutation-based Combinatorial Optimization*, Berlin, Heidelberg: Springer Berlin Heidelberg, Vol. 175, pp. 121-138, 2009a.

[31] F. Tasgetiren, Y.C. Liang, A. K. Pan, P. Suganthan, "Discrete/binary approach", *Differential Evolution: a Handbook for Global Permutation-based Combinatorial Optimization*. Berlin, Heidelberg: Springer Berlin Heidelberg, Vol. 175, pp. 139-162, 2009b.

[32] X. He, Q. Zhang, N. Sun, and Y. Dong, "Feature selection with discrete binary differential evolution", In: *Proc. of International Conf. on Artificial Intelligence and Computational Intelligence*. Vol. 4, pp. 327-330, 2009.

[33] H. Li, and L. Zhang, "A discrete hybrid differential evolution algorithm for solving integer programming problems", *Engineering Optimization*, Vol. 46, No. 9, pp. 1238-1268, 2014.

[34] M. Ali, S. M. Elsayed, T. Ray, and R. A. Sarker, "Memetic algorithm for solving resource constrained project scheduling problems", In: *Proc. of EEE Congress on Evolutionary Computation (CEC)*, Sendai, Japan, pp. 2761-2767, 2015.

[35] I.M. Ali, D. Essam, and K. Kasmarik, "A novel differential evolution mapping technique for generic combinatorial optimization problems", *Applied Soft Computing*, Vol. 80, pp. 297-309, 2019.

[36] A. P. Engelbrecht, J. Grobler, and J. Langeveld, "Set based particle swarm optimization for the feature selection problem", *Engineering Applications of Artificial Intelligence*, Vol. 85, pp. 324-336, 2019.

[37] A. Gain, and P. Dey, "Adaptive position-based crossover in the genetic algorithm for data clustering", *Recent Advances in Hybrid Metaheuristics for Data Clustering*, pp. 39-59, 2020.

[38] A.A. Chaves, M.G.C. Resende, M.J.A. Schuetz, J. K. Brubaker, H.G. Katzgraber, E.F.d. Arruda, and R. M. A. Silva, "A Random-Key Optimizer for Combinatorial Optimization", *arXiv preprint, arXiv:2411.04293*, 2024.

[39] A. A. Chaves, M.G.C. Resende, and R.M.A. Silva, "A random-key GRASP for combinatorial optimization", *arXiv preprint, arXiv:2405.18681*, 2024a.

[40] P. Singsathid, J. Wetweerapong, and P. Puphasuk, "Self-adaptive differential evolution algorithm with dynamic fitness-ranking mutation and pheromone strategy", *Bulletin of Electrical Engineering and Informatics*, Vol. 13, No. 1, pp. 559-571, 2024b.

[41] O. A. Alvrez-Flores, R. Rivera-Blas, L.A. Flores- Herrera, E.Z. Rivera-Blas, M. A. Funes-Lora, and P.A. Sino-Suarez, "Novel Modified Discrete Differential Evolution Algorithm to Solve the Operations Sequencing Problem in CAPP Systems", *Mathematics*, Vol. 12, No. pp. 1846, 2024.

[42] Y. Zhang, X. Liu, and P. Wang, "A hybrid differential evolution algorithm with adaptive neighborhood search for large-scale combinatorial optimization", *Information Sciences*, Vol. 658, No. 119873, 2024.

[43] J. Li, and R. Chen, "Improved discrete differential evolution algorithm with opposition-based learning for solving the travelling salesman problem", *Applied Soft Computing*, Vol. 139, No. 110747, 2023.

[44] G. Reinelt, "TSPLIB—A travelling salesman problem library", *ORSA journal on computing*, Vol. 3, No. 4, pp. 376-384, 1991.

[45] B. Liu, L. Wang, and Y. H. Jin, "An Effective PSO-Based Memetic Algorithm for Flow Shop Scheduling", *EEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, Vol. 37, No. 1, pp. 18-27, 2007.

[46] X. Li, and M. Yin, "A hybrid cuckoo search via Lévy flights for the permutation flow shop scheduling problem", *International Journal of Production Research*, Vol. 51, No. 16, pp. 4732-4754, 2013.